

A-Star Based Algorithm Applied to Target Search and Rescue by a UAV Swarm

Guilherme T. T. Bernardo, Luan M. B. Vogás, Sargon D. S. Rodrigues, Thiago G. G. Lopes, Cesar A. C. Marcondes, Denis S. Loubach, Elton F. Sbruzzi, Filipe A. N. Verri, Johnny C. Marques, Lourenço A. Pereira Jr., Marcos R. O. A. Maximo, and Vitor V. Curtis

Electronics and Computer Engineering Division

Aeronautics Institute of Technology

São José dos Campos, SP, Brazil

guilherme.bernardo@ga.ita.br, luan.vogas@ga.ita.br, sargon.rodrigues@ga.ita.br, and thiago.lopes.100797@ga.ita.br

Abstract—Drone swarms are being utilized in several areas, e.g., monitoring, surveillance, security, and search, demonstrating great potential. In this paper, we considered a specific number of drones flying simultaneously (using computational intelligence techniques between them to avoid collisions) and increased the effectiveness in conducting search tasks. The main contribution of this work is to propose a technique to coordinate the swarm using search methods based on the A* algorithm and its heuristics (i.e., Euclidean and Manhattan distances). The communication between drones is also evaluated and applied to a target search problem on a two-dimensional cost map. The results show a considerable optimization in the effectiveness of the drone swarm (fewer drones are blocked at local minima), which highlights the contribution of this work.

Index Terms—UAV Swarm, A-Star Algorithm, Target Search, Potential Fields, Swarm Intelligence, Collective Intelligence.

I. INTRODUCTION

In the last decades, significant changes have occurred in the life of human beings due to a new reality arising from the advances in information and communication technologies. It is noticed that the urban air traffic panorama has been increasingly explored through new studies that encompass Artificial Intelligence (AI) and Advanced Aerial Mobility (AAM), which includes, besides air cab services, delivery operations, image capture, and search and rescue with Unmanned Aerial Vehicles (UAVs), which have proven to be important for the integration of transportation systems in smart cities, delivery missions, and real-time detection of people and vehicles.

In this paper, we propose an application of the A* algorithm to optimize a simulated search for targets with UAV swarms. The open-source simulator in [1], which models the flight dynamics of UAVs with the *steering force* method and a potential field model, was enhanced to prevent UAVs from passing over no-fly zones and colliding with each other through a repulsive force nearby obstacles. Initially, each

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001 and by the Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP) - Grant 2021/06872-6.

drone has a route planned for the target search step. When the target is found by a drone, this information is shared with everyone and each one performs its own path planning mechanism (using the A*-based algorithm). Note that the simulation scenario is a continuous space environment and for the implementation of this technique, the map was discretized taking some assumptions presented in Section III.

The main contribution of this work is to optimize the collective intelligence of a UAV swarm based on the restrictions (e.g., obstacles and no-fly zones) by using the A*-based algorithm. Our technique improves the swarm intelligence model proposed in [1] so the drones do not get stuck in local minima. The proposed method is capable of enabling a more effective coordination between the UAVs, enabling a faster finding of the target in a discrete map used to evaluate the search strategy.

This paper is organized as follows. Section II details the theoretical background. Section III describes the problem formulation, while Section IV presents the comparative results. Finally, Section V presents the conclusions and future work.

II. BACKGROUND

This section provides a brief background on search algorithms, swarm intelligence, and the A* algorithm.

A. Search Algorithms and Swarm Intelligence

Search algorithms have been used for a variety of purposes such as searching and rescuing missing people in a forest, shipwrecks, or areas where the position of the target is unknown. In those cases, it is necessary to use communication techniques between the UAVs to optimize the time during the target search stage.

Currently, several models and studies are presented in well-established literature [2]–[5]. As mentioned by [1] and [2], the concepts and different ways of searching are already well addressed, however, there is still an opportunity for exploration when it comes to UAV swarms. In [3], a method was proposed for discretizing the area to be searched by dividing them into cells within a grid, where the passage of the UAV into each cell characterizes a searched area.

In [1], the authors proposed a solution for coordinating a UAV swarm using bivariate potential fields with autonomous and distributed intelligence between drones for a cooperative targeting application. In turn, when using an approach that applies AI, [6] applied decision tree and Particle Swarm Optimization (PSO) as an algorithm for parameter optimization, which allows decentralized decision-making for autonomous combat UAVs.

Using the methodology presented by [3], the problem to be studied is summarized in the use and evaluation of other search methods [2], [4] aiming to optimize the time required to visit each cell in a grid with the minimum time. Other papers, as [7], simulated a decentralized communication for UAVs swarm coordination through simple rules and local interactions. Hence, each UAV swarm could determine the map situation considering neighbors and objective positioning. To the best knowledge of the authors, this is the first study in the literature that attempts to understand the effects of an A* approach on a UAV swarm in target and search missions from an AI point of view. Previous work has not attempted to use AI local search algorithms to ensure the completion of the UAV swarm mission.

B. The A* Algorithm

Well known in the literature, the A* algorithm is a technique that combines the Dijkstra algorithm with an heuristic to guide the search, improving its performance [8]. This method uses an evaluation function based on the expression

$$f(v) = g(v) + h(v), \quad (1)$$

where $g(v)$ is the cost from the start to node v and $h(v)$ is the estimated cost of the optimal path from this same node v until the target node, calculated considering a heuristic of the area [9] (e.g., Manhattan, Euclidean or Chebyshev distance [10]). With expression (1), the value of $f(v)$ can be calculated for each of the adjacent nodes to v . Considering this information, the lowest cost path can be determined by choosing as the next node of the sequence the one with the smallest value of $f(v)$ [10].

For the implementation of the A* algorithm, usually two lists are needed: an OPEN and a CLOSED list. The OPEN list is a priority queue, and its purpose is to store the nodes that still have to be evaluated. The CLOSED list, on the other hand, stores the nodes that have already been evaluated [11]. A possible implementation for this algorithm is through the steps below [9], [11], [12]:

- 1) The OPEN list is initialized with the start node only and the CLOSED list must be empty.
- 2) Check the OPEN list. If it is empty, the algorithm terminates and returns an error message saying that it is not possible to reach the target.
- 3) The node with the smallest value $f(\cdot)$, calculated according to (1), is taken from the OPEN list and added to the CLOSED list and called v . If this node is the target, the search is terminated and the algorithm ends. The

optimal path can be obtained by following the sequence of parent nodes from the target to the beginning.

- 4) For the node v taken from the OPEN list in the previous step, if it is not the target, the successors, which are different from the ancestors, are determined. The result is grouped in a set.
- 5) If the successors found are neither in the OPEN nor the CLOSED list, they are added to the OPEN list, and the current node v is marked as its parent.
- 6) If there are nodes equal to successors in the OPEN list, their $g(\cdot)$ cost values should be updated if the $g(\cdot)$ cost of this particular successor is lower than the $g(\cdot)$ cost of the node already existing in OPEN. In this case, the parent of this successor is updated as the current v .
- 7) Return to step 2.

III. METHODOLOGY

A. Global Scenario

This paper is based on the simulation software developed in [1], which uses the concept of potential fields. The global scenario proposed is to test various search methods and make all the drones in the swarm rescue the target as soon as it is found. For calculation purposes, a field of 937500 m² (1250 x 750 m) was considered. The area was divided into blocks (cells) of 250 m² (50 x 50 m), providing a grid of 25 x 15 cells. Initially, the drones do not know the position of the target, however, once it is found, all the drones will know its position. The simulations performed have a maximum duration of 90 seconds. If the target is not found, or one of the drones is not able to reach it, the mission is considered a failure. The position of the target and the positions of the obstacles are randomly generated. To facilitate comparisons, each method was tested on the same simulation field. For more details, see the software stored in our GitHub repository^a.

B. Search Scenario

The focus of this paper is to evaluate both the search and rescue methods using a discretized algorithm in a continuous path planning context to obtain similar or better results when analyzing the total time for all drones within the swarm to reach the target.

When performing search operations, one of the key factors is the time to find the target, which is directly related to the technique applied for sweeping the area, which influences the number of cells to be visited. In this work, a random orientation was adopted for the drones, and the techniques presented in Fig. 1 were considered. As a form of evaluation, the search time and the time until all drones reach the target after it is found were checked.

We assumed a swarm with n drones, and a discretized area with N columns and M rows, where N and M are greater than n . Consequently, we have a grid with $G = N \cdot M$ cells. It is worth noting that after a drone finds the target, this position

^a<https://github.com/thiagolopes97/A-Star-Based-Algorithm-Applied-to-Target-Search-and-Rescue-by-an-UAV-Swarm/tree/main>

is shared with the entire swarm and each drone will trace an independent route to that point.

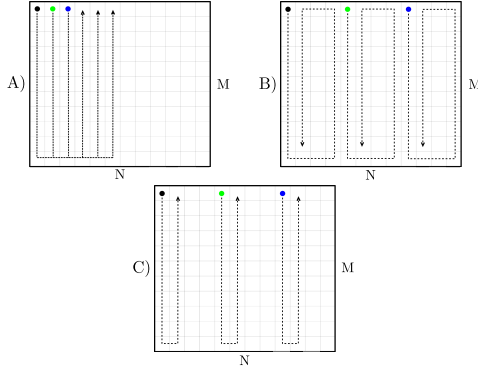


Fig. 1. A simple scheme of Search Pattern Scans used in this work: A) Grouped Columns, B) Inner Square Sparsed, and C) Columns Sparsed.

C. Heuristics

To perform a search using a swarm we need to use heuristics, since determining optimal paths tends to be infeasible due to the computational power involved [3]. In this work, the construction of heuristics was based on the concept of Euclidean and Manhattan distances, which are defined as:

- Euclidean distance: Let two points belonging to the Cartesian plane (x, y) be defined with coordinates (x_1, y_1) and (x_2, y_2) . The distance d between them is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}. \quad (2)$$

- Manhattan distance: Considering the same two points as before, the distance between them using this metric is:

$$d = |x_2 - x_1| + |y_2 - y_1|. \quad (3)$$

As an intermediate point of discretization, the center of the cell is used to evaluate the heuristic and compute the cost involved.

Normally, the A* method uses a 4- or 8-connected grid to perform a step-by-step movement. In this work, a methodology based on this method was used, which was applied to a continuous problem. The new technique consists of calculating the cost of all cells that form a block of 3 cells distant from the current position of the drone (i.e. 48 closest cells). This assumption was adopted so that the path planning and the time to reach the next point would be smaller. Fig. 2 illustrates the procedure.

In problems involving the A* algorithm, a cost function or cost map is usually defined. In this case, the discrete points of the map have an associated value [8]. In this work, to optimize the drone's path planning by making it avoid obstacles, two regions with a high cost were defined: the blue region in Fig. 3 shows prohibited zones, with dimensions equal to the radius of influence of the obstacle, while the yellow zone, being calculated using 10% more of this radius of influence, has a fixed cost of 10 units.

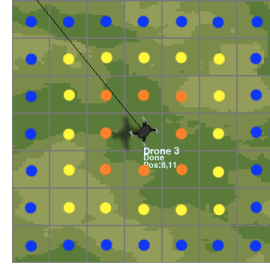


Fig. 2. Cells used in the cost calculation in the A*-based method. Orange, yellow, and blue dots represent the 3 cells distant from the current position of the drone.

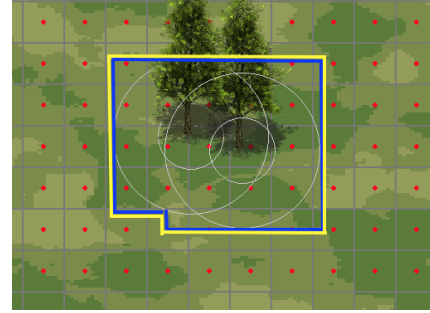


Fig. 3. Cost map: no-fly zone in blue and additional cost area in yellow.

D. Best Heuristic

To choose the best heuristic to improve the path planning performance (i.e. to avoid obstacles) and reach the target faster, three different distances were tested and used to calculate the path costs: Euclidean, Manhattan, and a sum of the two. Using the same scenario for each heuristic and search algorithm, the number of completed missions was calculated by evaluating the combinations between the number of drones and obstacles according to Table I.

TABLE I
HEURISTICS EXPERIMENTS.

Type	Drones	Obstacles
Euclidian (H1)	3,4,5,7	1,2,3,4,5,7,10
Manhattan (H2)	3,4,5,7	1,2,3,4,5,7,10
Euclidian + Manhattan (H3)	3,4,5,7	1,2,3,4,5,7,10

For each possible combination, 5 experiments were conducted. The computation of the final result was performed by taking the average of the times for each of the events. In cases where the target was not found or the drones failed to complete the mission in less than 90 seconds, this data was treated as missing. It is worth mentioning that the scenarios were randomly generated according to the number of drones and obstacles.

E. Continuous and Discrete Approach

To analyze the performance of each search method and the A*-based technique developed, it was assumed that all drones

in the swarm would start in search mode, searching for the target according to what was discussed in Section III-B. Once the target is found by any UAV of the swarm, this position is shared with all the other drones and rescue mode starts. In this mode, two policies were evaluated: Seek, a continuous technique, and the A*-based method, a discrete technique. The first one consists in going straight forward to the target position and the second one is the application of A* proposed in this paper. As Seek technique is a simple flight in a straight line to the target, there is no analysis of the obstacle that causes the collision or change of trajectory. Therefore, the main point of comparison between the two approaches concerns the number of times the UAV swarm gets stuck in local minima.

Given the performance evaluation, we applied a full factorial design for the experiments, varying the number of drones and obstacles, similar to that presented in Table I, for each of the 4 search methods: Random Search (RS), Grouped Columns (GC), Columns Sparsed (CS) and Inner Square Sparsed (ISS) using Seek and A*-based approaches, resulting in 224 possible combinations for five repetitions each, totaling 1120 executions. In the RS approach, the UAV swarm chooses a random cell within the grid and flies to it. Thus, UAVs are expected to collide in most tests in RS, as the choice of cells in the grid is arbitrary and multiple drones must compete for space for the same cell.

In this evaluation, two metrics were observed: time to find the target (i.e., search evaluation) and mission completion time (i.e., approach evaluation). As mentioned earlier, the mission is only considered a success when all drones locate the target. If in 90 seconds it has not been found, or drones are missing from the target circle, the mission is considered a failure.

IV. RESULTS AND DISCUSSIONS

A. Heuristic comparison

In Fig. 4 we have the average mission completion time in a bar graph comparing the 3 heuristics in the same search pattern and different scenarios.

In less dense environments (5 or fewer obstacles), all methods achieved the mission goal, i.e. all drones formed a swarm around the target. Such a result is a consequence of the position and configuration of the obstacles. As they become denser, reaching the target within the 90-second time becomes harder. In cases where the mission was a failure, no data were obtained for comparison. Analyzing the presented results, a similar time is observed for all three heuristics. In the case where three drones were employed, heuristics H2 and H3 were better compared to H1.

As the best heuristic, we selected H3 (i.e., the sum of the Euclidean and the Manhattan distances). Observing the simulation, it was possible to notice the formation of a simple obstacle avoidance system, where a common behavior is derived: a preference for horizontal or vertical actions to align with the nearest target axis is observed over a direct approach.

It is interesting to note that when using a larger number of obstacles (7 and 10 mostly), there were situations where the target was not found. Such a configuration of obstacles makes

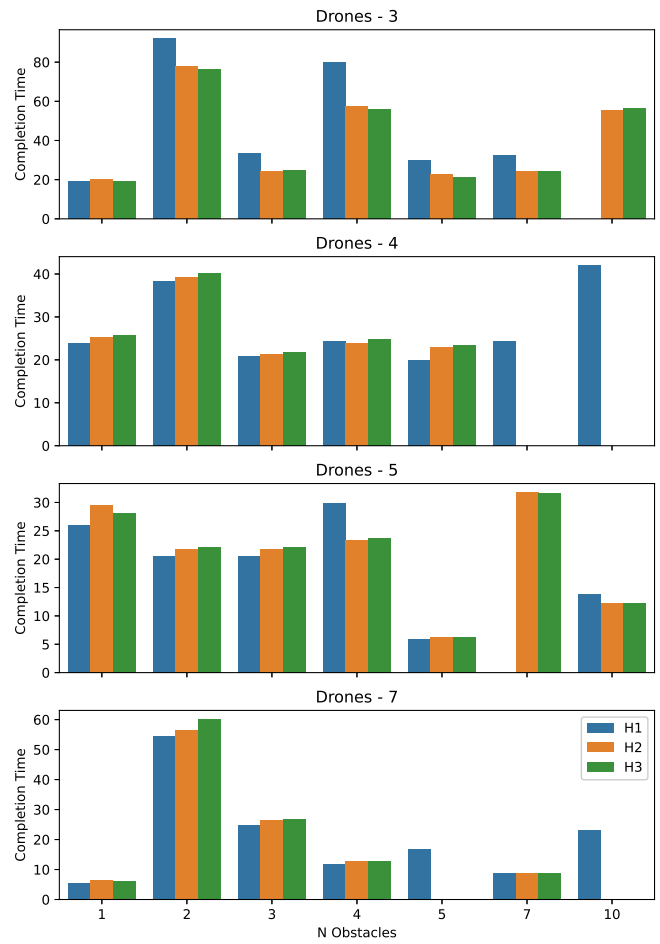


Fig. 4. Heuristic performance comparison with multiple drones.

it difficult to use the presented search schemes, resulting in mission failure.

B. Comparison of mission completion time in continuous and discrete scenarios

In Fig. 5 the average time spent in the simulation using both continuous and discrete approaches is presented. The bar graph represents the average time, while the lines represent the maximum and minimum time considering various search techniques.

In most cases, similar results were obtained, which is expected behavior in most simple scenarios. Another point analyzed was the mission completion time. Each method was tested in 112 different scenarios (number of drones, number and position of obstacles, and search pattern used). The continuous approach reached 101 times the goal (90.19 %), and the discrete approach 103 times (91.96 %), which is not statistically conclusive to say that one approach is better than the other. Fig. 6 shows the time required to complete the mission based on the search and approximation methods.

The results present in Table II, III, IV and V show the best search pattern and approach (i.e., Seek and A*-based

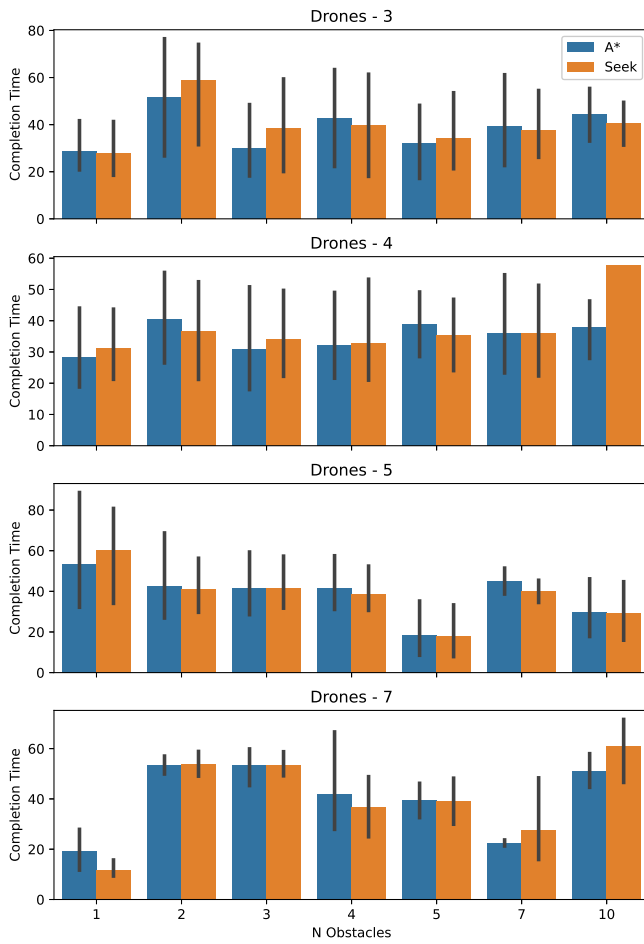


Fig. 5. Mission completion time using multiple drones and approaches.

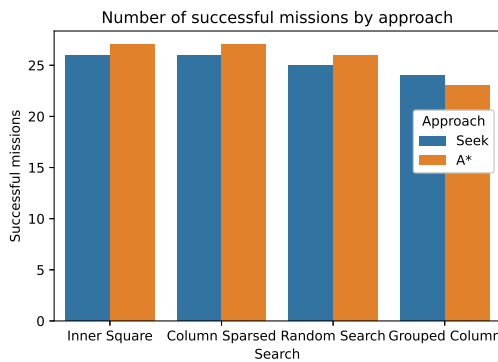


Fig. 6. Mission completion count by search and approach techniques.

methods) achieved for each number of obstacles. The last column refers to the time difference (Δ (s)) and shows the absolute difference in seconds of the test using the same search pattern and different approaches. The mission completing time (t (s)) is also shown.

Comparing the results using the open-source simulator, it is possible to notice that the best in most scenarios was the

TABLE II
BEST SEARCH AND APPROACH PERFORMANCE RESULTS USING 3 DRONES

Obstacles	Search Pattern	Approach	t (s)	Δ (s)
1	ISS	Seek	17.086	2.544
2	RS	A*	18.450	0.621
3	RS	A*	15.958	1.024
4	ISS	Seek	14.924	4.696
5	RS	A*	14.601	2.694
7	CS	A*	22.379	1.372
10	ISS	Seek	31.314	1.714

TABLE III
BEST SEARCH AND APPROACH PERFORMANCE RESULTS USING 4 DRONES

Obstacles	Search Pattern	Approach	t (s)	Δ (s)
1	RS	A*	17.334	2.603
2	RS	Seek	20.142	1.009
3	RS	A*	15.798	5.904
4	ISS	Seek	19.600	2.433
5	CS	Seek	21.813	6.436
7	ISS	Seek	2.880	1.350
10	RS	A*	27.938	11.276

TABLE IV
BEST SEARCH AND APPROACH PERFORMANCE RESULTS USING 5 DRONES

Obstacles	Search Pattern	Approach	t (s)	Δ (s)
1	CS	Seek	34.081	13.588
2	CS	Seek	29.141	1.360
3	CS	Seek	29.902	3.103
4	ISS	Seek	29.949	3.847
5	ISS	A*	6.945	0.099
7	ISS	Seek	34.519	4.152
10	ISS	Seek	12.954	1.767

TABLE V
BEST SEARCH AND APPROACH PERFORMANCE RESULTS USING 7 DRONES

Obstacles	Search Pattern	Approach	t (s)	Δ (s)
1	ISS	A*	9.260	2.307
2	ISS	Seek	49.050	7.978
3	ISS	A*	41.446	6.550
4	ISS	Seek	24.791	2.315
5	CS	Seek	30.406	2.780
7	ISS	Seek	15.660	6.914
10	GC	Seek	42.754	1.848

Seek approach. Statistically analyzing the difference in time between the methods, an average of 3.724 seconds was found, which represents 4.12 % of the total simulation time and a standard deviation of 3.162.

To complement the analysis and validate that drones are less

trapped in local minima using A*, tests were carried out in 3 scenarios (i.e., for 2, 3, and 4 drones) on maps generated with the number of obstacles ranging from 2 to 25. A total of 45 tests were performed, and 15 different obstacle arrangements were tested for each of the 3 scenarios. In the first scenario, where 2 drones were used on the map, the A* approach reached the target every time, bypassing the equipotential tree wall. In turn, when using the Seek approach with more than 12 obstacles, in all cases at least one of the drones gets stuck in a local minima.

In contrast, in the second scenario with 3 drones, A* and Seek performed the same up to 9 obstacles. When using 10 and 11 obstacles, only A* managed to reach the target. In the other cases, the three drones ended up disputing the same midpoint on the optimal path, which made them collide and impossible for them to reach the target. Something similar can be seen in the third scenario, so the Seek and A* performances were the same up to 9 obstacles. After that, Seek failed in all tests and A* managed to find the way only by using 10 obstacles.

The present model developed in this work is very useful to get out of local minima. When using the old Seek approach, in 40 % of the tests at least some drone gets stuck in local minima, compared to only 24.4 % with A*. In scenarios with 2 drones, the A* algorithm is enough to dribble the equipotential wall through a collision avoidance perspective. However, when using 3 and 4 drones in the simulation, due to a lack of communication between them, they end up disputing the same points. As A* is being applied by each drone independently, in some tests the movement of the drone interferes with the other, causing them to collide and preventing their passage.

V. CONCLUSION

In this paper, an A*-based algorithm was applied in a simulator which considers UAV swarms for search and rescue. The open-source software used was developed in [1]. The evaluated scenario concerns the search for a target in unknown locations with several obstacles represented through potential field simulation (i.e., trees and no-fly zones). As a contribution to the original paper and software, new search patterns were developed to test different search heuristics using a discrete approach based on the A* algorithm.

When evaluating the UAV swarms performance by varying the number of obstacles, we could observe that the divide and conquer pattern proves to be better in search cases. This type of pattern was used to divide the simulation grid into areas for each UAV so that each one of them could explore the environment through an intelligence based on local search. The evaluation of the completion time of each drone search task using an A* approach is very promising. For example, the previous steering behavior approach considers a step-by-step search model, which did not optimize the search time of the UAV swarms, resulting in a delayed task. The approach presented in this work recalculates the route at each iteration to avoid obstacles using A* in a discretized cost map. However, this methodology proves to be efficient as the drone movement remains continuous for using *steering force*. In this way, the

cost of each cell is calculated as the UAV swarm navigates the map using such a discrete methodology for path planning. Thus, even though the arrival time of the UAV swarm is similar to that observed without using A*, more drones reach the objective without getting stuck in obstacles.

For future work, different cost solutions can be tested to improve the results. Furthermore, new cost map heuristics could be addressed when using an algorithm that is not step-by-step based. In this sense, when a drone finds the target, we aim to apply A* to the other drones to determine the optimal path to the target. Since one drone has already reached the goal, the others do not necessarily need to continue exploring the map, being able to just follow the fastest path to the objective without colliding with obstacles. This form of connectivity between drones would improve the navigability model. Thus, the path can be followed continuously without interruption at middle points. Other metrics can also prove efficient in future evaluations, such as the number of cells visited during the search phase.

REFERENCES

- [1] L. Giacomossi, F. Souza, R. G. Cortes, H. Mirko Montecinos Cortez, C. Ferreira, C. A. C. Marcondes, D. S. Loubach, E. F. Sbruzzi, F. A. N. Verri, J. C. Marques, L. A. Pereira, M. R. O. A. Maximo, and V. V. Curtis, "Autonomous and collective intelligence for UAV swarm in target search scenario," in *2021 Latin American Robotics Symposium, 2021 Brazilian Symposium on Robotics, and 2021 Workshop on Robotics in Education*, 2021, pp. 72–77.
- [2] G. Skorobogatov, C. Barrado, and E. Salami, "Multiple UAV systems: A survey," *Unmanned Systems*, vol. 8, no. 02, pp. 149–169, 2020.
- [3] P. Garcia-Aunon and A. Barrientos Cruz, "Comparison of heuristic algorithms in discrete search and surveillance tasks using aerial swarms," *Applied Sciences*, vol. 8, no. 5, 2018.
- [4] A. C. Arbeit, "Adaptation and automation of search and rescue patterns with implementation for an operational unmanned aircraft system," Ph.D. dissertation, University of Washington, Washington, 2013.
- [5] R. D. Arnold, H. Yamaguchi, and T. Tanaka, "Search and rescue with autonomous flying robots through behavior-based cooperative intelligence," *Journal of International Humanitarian Action*, vol. 3, pp. 3–18, 2018.
- [6] L. Giacomossi, S. S. Dias, J. F. Brancalion, and M. R. Maximo, "Cooperative and decentralized decision-making for loyal wingman uavs," in *2021 Latin American Robotics Symposium, 2021 Brazilian Symposium on Robotics, and 2021 Workshop on Robotics in Education*. IEEE, 2021, pp. 78–83.
- [7] R. Demirci and E. Uslu, "Resultant force approach for swarm UAV behaviors," in *2021 Innovations In Intelligent Systems And Applications Conference*. IEEE, 2021, pp. 1–6.
- [8] Y. Zhou, X. Cheng, X. Lou, Z. Fang, and J. Ren, "Intelligent travel planning system based on A-star algorithm," in *2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control*, vol. 1, 2020, pp. 426–430.
- [9] J. Yao, C. Lin, X. Xie, A. J. Wang, and C.-C. Hung, "Path planning for virtual human motion using improved A* star algorithm," in *2010 Seventh International Conference on Information Technology: New Generations*, 2010, pp. 1154–1158.
- [10] F. Duchoň, A. Babinec, M. Kajan, P. Beňo, M. Florek, T. Fico, and L. Jurišica, "Path planning with modified A star algorithm for a mobile robot," *Procedia Engineering*, vol. 96, pp. 59–69, 2014, modelling of Mechanical and Mechatronic Systems.
- [11] I. S. AlShawi, L. Yan, W. Pan, and B. Luo, "Lifetime enhancement in wireless sensor networks using fuzzy approach and A-star algorithm," in *IET Conference on Wireless Sensor Systems*, 2012, pp. 1–6.
- [12] C. Ju, Q. Luo, and X. Yan, "Path planning using an improved A-star algorithm," in *2020 11th International Conference on Prognostics and System Health Management*, 2020, pp. 23–26.